

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale
Parte 3 – Prof. Luigi Di Caro

Panoramica dei 6 Progetti di Laboratorio

Dalla semantica lessicale su WordNet ai sistemi RAG ibridi:
teoria, scelte implementative e risultati principali

Gruppo di lavoro:

Studente	Matricola
Alessandro Olivero	915069
Samuele Iudice	1235245
Andrea Franco	1226739

(Lab 4 – Topic Modeling con BERTopic – svolto individualmente da Alessandro Olivero)

Docente: Prof. Luigi Di Caro
Anno Accademico: 2025/2026

1 Introduzione

Questo documento riassume, in forma compatta, i sei progetti di laboratorio della Parte 3 del corso. L'obiettivo non è ripetere le relazioni complete (già scritte per ciascun laboratorio), ma offrire un percorso unico e leggibile che mostri per ognuno: *quale problema teorico affronta, come è stato implementato* (con un estratto di codice realmente usato) e *cosa hanno mostrato i risultati*.

I sei progetti tracciano un arco coerente: si parte dalla **semantica lessicale simbolica** basata su WordNet e sulle definizioni (Lab 1–3), si passa alla **semantica distribuzionale neurale** con gli embedding e il topic modeling (Lab 4), e si arriva ai **sistemi di generazione aumentata da retrieval** (Lab 5 e il Progetto Estesio), dove semantica simbolica (BM25, matching lessicale) e semantica neurale (embedding densi) tornano a incontrarsi nello stesso sistema.

#	Progetto	Tecnica centrale	Risultato chiave
1	Similarità su definizioni	Jaccard lessicale vs cosine SBERT	$\Delta(\text{Simsem}-\text{Simlex}) \approx 0.27$ sempre
2	Content-to-Form (WordNet)	Genus-extraction + iponimi + SBERT	accuracy 1.8%, ma cond. 50%
3	Polisemia cross-lingue	WordNet/OMW multilingue + entropia	EN (22.95) $\gg$ ES (9.90) > IT (7.15) sensi medi
4	Topic Modeling	BERTopic (embedding+UMAP+HDBSCAN)	modello $\rightarrow$ velocità, clustering $\rightarrow$ purezza
5	RAG semantico	SciBERT + FAISS + Phi-3.5-mini	crolla su corpus grandi (P@5=0.04)
6	RAG ibrido (estesio)	+ BM25 + Reciprocal Rank Fusion	P@5 0.04 $\rightarrow$ 0.48 sullo stesso caso

2 Lab 1 – Similarità Lessicale e Semantica su Definizioni

Cornice teorica

Due definizioni dello stesso concetto possono condividere *molte parole* (similarità lessicale, o *forma*) oppure *poche parole ma lo stesso significato* (similarità semantica, o *contenuto*) – è la distinzione content vs. form che attraversa tutto il laboratorio successivo (Lab 2). La similarità lessicale si misura con l'indice di **Jaccard** $J(A, B) = |A \cap B| / |A \cup B|$ su insiemi di lemmi; quella semantica con la **cosine similarity** tra embedding prodotti da un modello Sentence-BERT, addestrato a mappare frasi semanticamente vicine in vettori vicini indipendentemente dalle parole usate.

Cosa è stato fatto. Sono state raccolte definizioni in linguaggio naturale per 4 concetti scelti per incrociare due assi: **astratto/concreto** $\times$ **generico/specifico** (*Music*, *Ethics*, *Tree*, *Teapot*). Ogni definizione viene ripulita con una pipeline unica (minuscolo, rimozione punteggiatura/stopword, lemmatizzazione POS-aware); su questi lemmi si calcola **Simlex** come media delle similarità di Jaccard a coppie. Per **Simsem** si usa invece il testo *grezzo* (non ripulito) codificato con SBERT (all-MiniLM-L6-v2): rimuovere stopwords e sintassi danneggerebbe l'input rispetto a come il modello è stato addestrato.

Pipeline di preprocessing e le due metriche di similarità

```
1 def preprocess_definition(text):
2     text = text.lower()
3     text = re.sub(f"[{re.escape(string.punctuation)}]", " ", text)
4     tokens = [t for t in text.split() if t.isalpha()
5               and t not in STOP_WORDS and len(t) > 2]
6     tagged = pos_tag(tokens)
7     lemmas = [LEMMATIZER.lemmatize(tok, _wordnet_pos(pos))
8               for tok, pos in tagged]
9     return " ".join(lemmas), set(lemmas)
10
11 def jaccard_similarity(set1, set2):
12     if not set1 or not set2:
13         return 0.0
14     return len(set1 & set2) / len(set1 | set2)
15
16 # Simsem: cosine similarity su embedding SBERT del testo NON ripulito
17 embs = SBERT_MODEL.encode(raw_for_sbert)
18 cos_sim_matrix = cosine_similarity(embs)
```

Risultati. La similarità semantica supera sempre, e di molto, quella lessicale:

Aggregazione	Simlex	Simsem	Δ
Concetti astratti	0.046	0.314	0.267
Concetti concreti	0.074	0.330	0.256
Definizioni generiche	0.065	0.341	0.276
Definizioni specifiche	0.055	0.302	0.247

Il divario ($\Delta \approx 0.25$ – 0.28) resta pressoché costante in tutte le suddivisioni: le persone descrivono lo stesso concetto con parole molto diverse (basso overlap lessicale), ma SBERT cattura comunque il significato condiviso. Gli scarti tra le sottocategorie sono invece piccoli: i concetti concreti hanno una similarità leggermente più alta di quelli astratti (definizioni più vincolate a un referente fisico condiviso), e le definizioni generiche leggermente più alte di quelle specifiche.

3 Lab 2 – Content-to-Form: Ricerca Onomasiologica su WordNet

Cornice teorica

La **semasiologia** va dalla parola al significato (i laboratori classici di WSD); l'**onomasiologia** fa il percorso inverso: dato un significato (una definizione), trovare la parola/il synset che lo esprime – esattamente il compito “content-to-form”. Il principio lessicografico del **genus-differentia** (Aristotele) dice che una buona definizione nomina prima il **genus** (l'iperonimo immediato, es. “a container...”) e poi lo differenzia dagli altri membri della stessa categoria.

Cosa è stato fatto. `extract_genus` individua il genus con una regola sintattica (il nome subito dopo “is/are”) con fallback sul primo sostantivo della definizione. A partire dal genus si genera l'insieme dei **candidati** scendendo negli iponimi di WordNet fino a profondità 3 (con un cap di 300, altrimenti un genus troppo generico come “object” farebbe esplodere la ricerca). I candidati vengono infine ordinati per similarità (SBERT o TF-IDF) tra la definizione originale e la gloss di ciascun candidato.

Estrazione del genus e generazione dei candidati dagli iponimi

```
1 def extract_genus(definition):
2     # Regola 1: il sostantivo subito dopo "is/are"
3     m = re.search(r"\b(?:is|are)\b\s+(?:an?\s+)?([a-zA-Z-]+)",
4                   definition.lower())
5     if m and wn.synsets(m.group(1), pos=wn.NOUN):
6         return m.group(1)
7     # Regola 2 (fallback): primo sostantivo della definizione
8     tagged = pos_tag([t for t in word_tokenize(definition.lower())
9                       if t.isalpha()])
10    for tok, pos in tagged:
11        if pos.startswith("NN") and wn.synsets(tok, pos=wn.NOUN):
12            return tok
13    return None
14
15 def get_candidates_from_genus(genus, depth=3, max_candidates=300):
16     unique = []
17     for syn in wn.synsets(genus, pos=wn.NOUN):
18         unique.append(syn)
19         for hypo in syn.closure(lambda s: s.hyponyms(), depth=depth):
20             unique.append(hypo)
21             if len(unique) >= max_candidates:
22                 return unique
23     return unique
```

Risultati. L'accuracy globale è **1.8%** – un numero che, isolato, sembrerebbe un fallimento. Scomponendolo si trova il risultato più interessante del laboratorio: la **coverage** (quante volte il synset target è anche solo presente tra i candidati generati) è del **3.6%** – è questo il vero collo di bottiglia, non il ranking. Infatti, condizionando sui soli casi in cui il target è raggiungibile, l'accuracy sale al **50%**: quando SBERT ha la possibilità di scegliere il synset giusto, lo fa in metà dei casi. Una versione precedente con 5 glosse pre-selezionate a mano (target sempre incluso) otteneva 32–89% di accuracy: un compito artificialmente facile, perché il retrieval era già risolto in partenza. La ricerca onomasiologica non vincolata su tutto WordNet è quindi un compito genuinamente difficile, e la scomposizione coverage/accuracy-condizionata separa in modo pulito l'errore di *retrieval* da quello di *ranking*.

4 Lab 3 – Polisemia e Asimmetria Cross-linguistica

Cornice teorica

Non esiste una “polisemia universale”: la stessa area concettuale può essere lessicalizzata in modo diverso da lingua a lingua (es. l'inglese *bank* copre sia “istituto finanziario” sia “sponda del fiume”, l'italiano li separa in *banca/banco*, mentre lo spagnolo concentra più sensi in *banco*). Contare i sensi di WordNet non basta a misurare l'ambiguità realmente percepita: due lemmi con lo stesso numero di sensi possono avere un uso concentrato su un solo senso dominante o distribuito uniformemente su tutti – distinzione che l'**entropia di Shannon** $H = -\sum_i p_i \log_2 p_i$ sulla distribuzione di frequenza d'uso dei sensi cattura, mentre il semplice conteggio no.

Cosa è stato fatto. 20 lemmi (nomi, verbi, aggettivi) allineati manualmente tra inglese, italiano e spagnolo (es. *bank/banca/banco*); per ciascuno si contano i sensi con filtro di categoria grammaticale via WordNet (EN) e Open Multilingual WordNet (IT/ES). Per l'inglese l'entropia è calcolata dalle frequenze d'uso reali in SemCor (quante volte ciascun senso compare in un corpus annotato); per italiano e spagnolo, in assenza di dati di frequenza in OMW, si usa una distribuzione uniforme come fallback esplicito (limite dichiarato, non nascosto).

Entropia reale (SemCor) per l'inglese, fallback uniforme per IT/ES

```

1 def calcola_metriche(synsets, lemma, lang_code):
2     n = len(synsets)
3     if lang_code == "eng":
4         counts = np.array(english_sense_counts(lemma, synsets))
5         if counts.sum() > 0:
6             dist = counts / counts.sum() # frequenza reale
7             ent = float(scipy_entropy(dist, base=2))
8             return {"entropia": ent, "metodo_entropia": "real_semcor"}
9     dist = np.ones(n) / n # fallback uniforme
10    return {"entropia": float(scipy_entropy(dist, base=2)),
11            "metodo_entropia": "uniform_fallback"}

```

Risultati. Gerarchia sistematica e netta: **22.95** sensi medi in inglese, contro **9.90** in spagnolo e **7.15** in italiano, con asimmetrie individuali fino a **9.83×** (*break*, EN/IT). I verbi mostrano l'asimmetria più marcata (39.33 EN vs 11.83 IT), mentre i sostantivi concreti con referente fisico stabile (*mouth*, *table*, *foot*) mostrano la minore. Un dato che qualifica tutto il resto: la correlazione tra entropia reale e $\log_2(n_sensi)$ è $r = 0.76$, non 1 – lemmi come *head* (33 sensi, ma 82% dell'uso concentrato su un solo senso) sono formalmente molto polisemici ma praticamente quasi monosemici in uso, mentre altri (*take*, *break*) sono genuinamente ambigui con uso distribuito. Contare i sensi, da solo, sovrastima l'ambiguità reale.

5 Lab 4 – Topic Modeling con BERTopic

Cornice teorica

BERTopic scompone il topic modeling in una pipeline modulare a quattro stadi: **embedding** (ogni documento diventa un vettore denso), **UMAP** (riduzione dimensionale che preserva la struttura locale dei vicini), **HDBSCAN** (clustering density-based, che a differenza di K-Means non impone il numero di cluster e produce esplicitamente una classe di *outlier* per i punti in zone a bassa densità), e infine **c-TF-IDF** (una variante del TF-IDF calcolata per cluster invece che per documento, usata per estrarre le parole più rappresentative di ogni topic).

Cosa è stato fatto. Su 44949 abstract NLP da arXiv sono state confrontate 3 configurazioni cambiando **un solo parametro alla volta** rispetto alla baseline, per poter attribuire ogni differenza a una causa precisa: Config 1 (baseline, embedding **gte-small**); Config 2 (solo il modello di embedding cambia, in **all-MiniLM-L6-v2**); Config 3 (stesso embedding di Config 2, cambiano solo i parametri di UMAP/HDBSCAN verso una granularità più grossolana).

Pipeline BERTopic: embedding + UMAP + HDBSCAN

```

1 umap_model = UMAP(**config["umap_params"])
2 hdbscan_model = HDBSCAN(**config["hdbscan_params"])
3
4 topic_model = BERTopic(
5     embedding_model=emb_model,
6     umap_model=umap_model,
7     hdbscan_model=hdbscan_model,
8 )
9 topics, probs = topic_model.fit_transform(docs, embeddings)

```

Risultati.

Configurazione	# Topic	% Outlier	Tempo totale
Config 1 – baseline (gte-small)	151	31.9%	7 417 s ($\approx 2h$)
Config 2 – solo modello (MiniLM)	116	27.5%	513 s
Config 3 – solo clustering (grossolano)	46	33.5%	35 s

Isolare le due variabili rivela che agiscono su assi indipendenti: cambiare *solo* il modello di embedding (1→2) governa soprattutto **velocità** (14.5× più veloce) e in parte granularità (−23% topic); cambiare *solo* i parametri di clustering (2→3) governa soprattutto **granularità** (−60% topic) e **purezza semantica** – la Config 3 mostra infatti stopwords (*the, and, to*) che contaminano le parole chiave di più topic, un failure mode assente nelle altre due configurazioni. La Config 2 risulta il miglior compromesso: meno outlier della baseline, tempo pratico su CPU, nessuna contaminazione.

6 Lab 5 – Sistema RAG su Articoli Scientifici

Cornice teorica

Un LLM ha conoscenza *parametrica*, fissata al training e soggetta ad allucinazioni quando gli si chiede di citare fatti specifici. Il **Retrieval-Augmented Generation** (RAG) affianca un **retriever** che recupera documenti rilevanti da una collezione esterna e li inserisce nel prompt, così il generatore condiziona la risposta su testo verificabile invece che sulla sola memoria interna. Il retrieval **semantico** (denso) codifica query e documenti nello stesso spazio di embedding e cerca per similarità coseno/prodotto scalare – qui tramite **FAISS** con ricerca esatta.

Cosa è stato fatto. Corpus di paper NLP da arXiv (fino a 44 949 documenti), embedding con **SciBERT** (pre-addestrato su 1.14M paper scientifici, quindi più adatto di un modello generico a questo dominio), indice FAISS **IndexFlatIP**, generazione con **Phi-3.5-mini-instruct** (decoding greedy per risposte riproducibili). L’esperimento fa variare sia la dimensione del corpus $N_DOCS \in \{1000, \dots, 44949\}$ (sottoinsiemi annidati, per isolare l’effetto della dimensione dal campionamento casuale) sia k , misurando Precision@ k , Context Relevance, Answer Relevance e una Faithfulness proxy lessicale.

Retrieval semantico: query e corpus nello stesso spazio di embedding

```
1 def retrieve(query, faiss_index, documents, metadata, k=5):
2     query_emb = embedding_model.encode(
3         [query], normalize_embeddings=True
4     ).astype(np.float32)
5     similarities, indices = faiss_index.search(query_emb, k)
6     return [{'rank': i + 1, 'similarity': float(sim),
7             'document': documents[idx], 'metadata': metadata[idx]}
8             for i, (sim, idx) in enumerate(zip(similarities[0], indices[0]))]
```

Risultati. La Precision@ k media resta stabile tra 0.24 e 0.29 su tutte le dimensioni di corpus – in apparenza un sistema robusto. Ma scomponendo per singola query emerge il limite reale: sulla query “*What is BERT and how does self-attention work?*”, la Precision@ k crolla quasi a **zero** sui corpus grandi (il paper specifico su BERT viene “affogato” fra migliaia di vicini semantici simili – altri paper su transformer, attention, pre-training). È un limite strutturale del retrieval puramente semantico su corpus ampi, e motiva direttamente l’estensione ibrida del progetto successivo.

7 Progetto Esteso – Hybrid Search RAG

Cornice teorica

Il retrieval denso è robusto a parafrasi e sinonimi ma può perdere la corrispondenza *esatta* di un termine specifico (un acronimo come “BERT”) quando il corpus è grande e pieno di vicini semantici. Il retrieval **lessicale** (**BM25**, evoluzione probabilistica del TF-IDF) fa l’opposto: preciso sul matching esatto dei termini, cieco alla parafrasi. La **Reciprocal Rank Fusion** (RRF) combina le due liste ordinate senza dover calibrare punteggi eterogenei (similarità coseno vs. score BM25, scale non comparabili): assegna a ogni documento $1/(k_{\text{rrf}} + \text{rank})$ in ciascuna lista in cui compare, e somma i contributi.

Cosa è stato fatto. Stesso corpus ed embedding SciBERT di Lab 5 (cache condivisa, per evitare di ricodificare 44949 abstract due volte), a cui si affianca un indice BM25. `retrieve_hybrid` recupera i primi 20 risultati da entrambi i canali (semantico e lessicale) e li fonde con RRF. Il confronto base-vs-ibrido è sistematico: stesse query, stesso *N_DOCS*, stesse metriche di Lab 5, più una misura di **robustezza** (overlap tra i risultati di una query e delle sue parafrasi).

Reciprocal Rank Fusion: unire due ranking senza calibrare i punteggi

```

1 def reciprocal_rank_fusion(results_list, k=60):
2     rrf_scores, doc_map = {}, {}
3     for results in results_list:
4         for item in results:
5             ix = item['idx']
6             rrf_scores[ix] = rrf_scores.get(ix, 0) + 1 / (k + item['rank'])
7             doc_map[ix] = item
8     sorted_docs = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
9     return [{'rank': i + 1, 'rrf_score': s, 'idx': ix,
10             'document': doc_map[ix]['document']}
11             for i, (ix, s) in enumerate(sorted_docs)]
12
13 def retrieve_hybrid(query, faiss_index, bm25, documents, metadata, k=5):
14     sem = retrieve_semantic(query, faiss_index, documents, metadata, k=20)
15     bm = retrieve_bm25(query, bm25, documents, metadata, k=20)
16     return reciprocal_rank_fusion([sem, bm])[:k]

```

Risultati. Il miglioramento è sistematico e in un caso drammatico:

N_DOCS	P@5 base	P@5 ibrido	Faithfulness base	Faithfulness ibrido
1 000	0.16	0.48	0.568	0.765
5 000	0.28	0.68	0.613	0.719
15 000	0.20	0.56	0.655	0.755
25 000	0.20	0.56	0.628	0.722
44 949	0.04	0.48	0.496	0.669

Il caso più importante di tutto l'esperimento è l'ultima riga: sul corpus completo, il sistema base **crolla** (P@5=0.04, quasi zero) mentre l'ibrido **tiene** (P@5=0.48, 12× meglio) – BM25 recupera esattamente il paper su “BERT” che il solo retrieval semantico perde tra migliaia di vicini. Anche la Faithfulness migliora in ogni configurazione (+0.09 a +0.20). L'unica metrica che non premia sempre l'ibrido è la robustezza alle parafrasi, migliore solo per il corpus più piccolo – aggiungere BM25 aiuta la precisione ma non garantisce automaticamente più stabilità a piccole variazioni della query.

8 Conclusioni

I sei progetti, pur indipendenti, condividono un filo metodologico: non fermarsi al numero aggregato, ma **scomporlo** per capire cosa lo determina davvero. Lab 1 scompone la similarità in lessicale/semantica; Lab 2 scompone l'errore in retrieval (coverage) vs. ranking (accuracy condizionata); Lab 3 scompone il conteggio dei sensi in entropia d'uso reale; Lab 4 isola un parametro alla volta per attribuire ogni effetto a una causa precisa; Lab 5 e il Progetto Esteso confrontano sistematicamente base e ibrido sulle stesse query e sullo stesso corpus, invece di riportare solo la configurazione migliore. È questa disciplina di confronto controllato – più che la scelta di un singolo modello o iperparametro – il contributo metodologico comune a tutto il lavoro.